Министерство науки и высшего образования РФ
Федеральное государственное автономное
образовательное учреждение высшего образования
«СИБИРСКИЙ ФЕДЕРАЛЬНЫЙ УНИВЕРСИТЕТ»

Институт космических и информационных технологий
институт

Программная инженерия
кафедра

ОТЧЕТ О ПРАКТИЧЕСКОЙ РАБОТЕ №2

по дисциплине «Серверное программирование»

Тема: связи между таблицами и генерация данных

Преподаватель _____ _____ _____
подпись, дата | инициалы, фамилия

Студент _____ _____ Mohamed Amini
номер группы, зачетной книжки | подпись, дата | инициалы, фамилия

Красноярск 2026

1 Цель и постановка задачи

Цель работы – реализовать связи между таблицами базы данных «ВелоГрад», спроектированной в практическом задании № 1, и сгенерировать большой объём тестовых данных. Связи должны соответствовать разработанной модели.

Требования задания:

– проанализировать модель и определить тип связи для каждой пары таблиц;

– реализовать связи: внешние ключи (FK), ограничения UNIQUE, промежуточные таблицы для связей «многие-ко-многим»;

– продемонстрировать соединения INNER, LEFT, FULL OUTER и CROSS JOIN;

– сгенерировать не менее 10 000 записей в одной или нескольких таблицах;

– выполнить типовой SQL-запрос приложения с использованием связей.

Критерии приёмки: внешние ключи и ограничения целостности в DDL; примеры разных JOIN с пояснением; не менее 10 000 сгенерированных записей; типовой запрос приложения. Отчёт содержит SQL-скрипты, план генерации, результаты JOIN и типового запроса.

2 Стек и запуск

Практическое задание № 1 выполнено на SQLite; для заданий № 2 и № 3 выбрана СУБД PostgreSQL 17 – референсная СУБД задания, в которой есть генератор рядов generate_series, полноценный FULL OUTER JOIN и план выполнения EXPLAIN ANALYZE. Программа на TypeScript (Node.js 24, без шага сборки) запускает сервер и последовательно выполняет SQL-скрипты клиентом psql. PostgreSQL 17.11 развёрнут отдельным кластером в каталоге .pgdata, слушает адрес 127.0.0.1 на порту 54329; база данных velograd создана в кодировке UTF8.

Запуск: npm run pg:start (старт сервера), затем npm start. Программа выполняет скрипты, перечисленные в таблице 1.

Таблица 1 – Скрипты работы

Файл	Назначение
sql/01_schema.sql	создание таблиц базы «ВелоГрад» в PostgreSQL (схема ПЗ №1)
sql/02_seed_base.sql	базовые данные ПЗ №1 (от 10 до 16 строк в таблице)
sql/03_relations.sql	новые связи: рекурсивная связь клиентов, кошелёк клиента, избранные станции
sql/04_joins.sql	соединения INNER, LEFT, FULL OUTER и CROSS JOIN
sql/05_generate.sql, sql/generate.sql	массовая генерация данных
sql/06_typical_query.sql	типовой запрос приложения

3 Анализ и реализация связей

3.1 Типы связей и действия при удалении

Для каждой пары таблиц определён тип связи, затем выбрано действие при удалении родительской строки (ON DELETE). Результат приведён в таблице 2.

Таблица 2 – Связи между таблицами

Связь	Тип	Реализация	ON DELETE
Район – Станция	1 : N	station.district_id → district	RESTRICT
Станция – Велосипед	1 : N	bicycle.home_station_id → station	RESTRICT
Клиент, Велосипед, Тариф – Поездка	1 : N	rental.client_id, bicycle_id, tariff_id	RESTRICT
Станция – Поездка	1 : N (две связи)	rental.start_station_id, end_station_id	RESTRICT
Поездка – Платёж	1 : 1	payment.rental_id: FK и UNIQUE	CASCADE
Клиент – Кошелёк	1 : 1	client_wallet.client_id: PRIMARY KEY и FK одновременно	CASCADE
Клиент – Станция (избранное)	M : N	client_favorite_station: два FK, составной PRIMARY KEY	CASCADE
Клиент – Клиент (кого пригласил)	1 : N, рекурсивная	client.referred_by → client	SET NULL

Выбор действия при удалении: RESTRICT – для справочников и истории (нельзя удалить район, станцию, тариф, велосипед или клиента, на которых ссылаются поездки); CASCADE – для зависимых данных без самостоятельного смысла (платёж удалённой поездки, кошелёк и избранные станции удалённого клиента); SET NULL – для «мягкой» ссылки: запись приглашённого клиента сохраняется, ссылка на пригласившего исчезает.

Связь между таблицами rental и station двойная – станция начала и станция конца поездки; конечная станция допускает NULL, пока поездка не завершена. Связь «клиент – велосипед» тоже «многие-ко-многим», но реализована таблицей rental с собственными атрибутами (время, стоимость, расстояние).

3.2 Реализация связей

Связи, существовавшие в схеме ПЗ №1, заданы в скрипте 01_schema.sql ограничениями FOREIGN KEY; в качестве примера – таблица rental с пятью внешними ключами:

```
CREATE TABLE rental (
    id               integer GENERATED BY DEFAULT AS IDENTITY PRIMARY KEY,
    client_id        integer       NOT NULL,
    bicycle_id       integer       NOT NULL,
    tariff_id        integer       NOT NULL,
    start_station_id integer       NOT NULL,
    end_station_id   integer       NULL,
    started_at       timestamp     NOT NULL,
    finished_at      timestamp     NULL,
    distance_km      numeric(7, 2) NULL CHECK (distance_km IS NULL OR distance_km >= 0),
    total_cost       numeric(10, 2) NULL CHECK (total_cost IS NULL OR total_cost >= 0),

    CONSTRAINT ck_rental_finished CHECK (finished_at IS NULL OR finished_at > started_at),

    CONSTRAINT ck_rental_complete CHECK (
        (finished_at IS     NULL AND end_station_id IS     NULL AND total_cost IS     NULL) OR
        (finished_at IS NOT NULL AND end_station_id IS NOT NULL AND total_cost IS NOT NULL)
    ),

    CONSTRAINT fk_rental_client  FOREIGN KEY (client_id)        REFERENCES client  (id) ON DELETE
RESTRICT,
    CONSTRAINT fk_rental_bicycle FOREIGN KEY (bicycle_id)       REFERENCES bicycle (id) ON DELETE
RESTRICT,
    CONSTRAINT fk_rental_tariff  FOREIGN KEY (tariff_id)        REFERENCES tariff  (id) ON DELETE
RESTRICT,
    CONSTRAINT fk_rental_start   FOREIGN KEY (start_station_id) REFERENCES station (id) ON DELETE
RESTRICT,
    CONSTRAINT fk_rental_end     FOREIGN KEY (end_station_id)   REFERENCES station (id) ON DELETE RESTRICT
);
```

Новые связи добавляет скрипт 03_relations.sql: рекурсивная связь «клиент – приглашённый клиент» (ALTER TABLE с проверкой, что клиент не

приглашает сам себя), связь «один-к-одному» «клиент – кошелёк» (первичный ключ совпадает с внешним, поэтому UNIQUE не нужен) и связь «многие-ко-многим» «клиент – избранная станция» (промежуточная таблица с двумя внешними ключами и составным первичным ключом, запрещающим дубликаты пар). Таблицы заполняются тестовыми данными.

```
ALTER TABLE client
    ADD COLUMN referred_by integer NULL
        CONSTRAINT fk_client_referred_by REFERENCES client (id) ON DELETE SET NULL;

ALTER TABLE client
    ADD CONSTRAINT ck_client_not_self_referral CHECK (referred_by IS NULL OR referred_by <> id);

UPDATE client SET referred_by = 1 WHERE id IN (4, 7, 9);
UPDATE client SET referred_by = 4 WHERE id = 11;

CREATE TABLE client_wallet (
    client_id  integer        PRIMARY KEY,
    balance    numeric(10, 2) NOT NULL DEFAULT 0 CHECK (balance >= 0),
    updated_at timestamp      NOT NULL DEFAULT now(),

    CONSTRAINT fk_wallet_client FOREIGN KEY (client_id)
        REFERENCES client (id) ON DELETE CASCADE
);

INSERT INTO client_wallet (client_id, balance)
SELECT id, 100 + (id * 37) % 900 FROM client;

CREATE TABLE client_favorite_station (
    client_id  integer NOT NULL,
    station_id integer NOT NULL,
    added_on   date    NOT NULL DEFAULT current_date,

    CONSTRAINT pk_client_favorite_station PRIMARY KEY (client_id, station_id),
    CONSTRAINT fk_favorite_client  FOREIGN KEY (client_id)  REFERENCES client  (id) ON DELETE
CASCADE,
    CONSTRAINT fk_favorite_station FOREIGN KEY (station_id) REFERENCES station (id) ON DELETE CASCADE
);

INSERT INTO client_favorite_station (client_id, station_id, added_on)
SELECT c.id, s.id, date '2025-01-01' + (c.id * 5 + s.id) % 300
FROM client AS c
JOIN station AS s ON (c.id * 7 + s.id * 3) % 5 = 0
WHERE s.id <= 8;
```

Сводка ограничений схемы по системному каталогу PostgreSQL приведена на рисунках 1 и 2: 9 первичных ключей, 12 внешних ключей, 8 ограничений UNIQUE и 23 ограничения CHECK.

```
psql -d velograd -c "SELECT contype, count(*) FROM pg_constraint WHERE connamespace =
'public'::regnamespace GROUP BY contype"
 contype | count
---------+-------
 c       |    23
 f       |    12
 p       |     9
 u       |     8
(4 rows)
```

Рисунок 1 – Число ограничений по видам (p – PRIMARY KEY, f – FOREIGN KEY, u – UNIQUE, c – CHECK)

```
psql -d velograd -c "SELECT conrelid::regclass, conname, confrelid::regclass, <действие> FROM
pg_constraint WHERE contype = 'f'"
          table          |        conname        | references_table | on_delete
-------------------------+-----------------------+------------------+-----------
 station                 | fk_station_district   | district         | RESTRICT
 bicycle                 | fk_bicycle_station    | station          | RESTRICT
 client                  | fk_client_referred_by | client           | SET NULL
 rental                  | fk_rental_bicycle     | bicycle          | RESTRICT
 rental                  | fk_rental_client      | client           | RESTRICT
 rental                  | fk_rental_end         | station          | RESTRICT
 rental                  | fk_rental_start       | station          | RESTRICT
 rental                  | fk_rental_tariff      | tariff           | RESTRICT
 payment                 | fk_payment_rental     | rental           | CASCADE
 client_wallet           | fk_wallet_client      | client           | CASCADE
 client_favorite_station | fk_favorite_client    | client           | CASCADE
 client_favorite_station | fk_favorite_station   | station          | CASCADE
(12 rows)
```

Рисунок 2 – Внешние ключи и действие при удалении

4 Демонстрация соединений (JOIN)

Соединения демонстрируются на базовых данных ПЗ №1: 10 районов, 12 станций и 16 поездок, – поэтому видно, какие строки попадают в результат. В данных заложены особенности: район «Красногорский» без станций, станция «Заречье», на которую никто не приезжал, и две незавершённые поездки. Скрипт 04_joins.sql содержит по одному запросу каждого вида.

4.1 INNER JOIN

Возвращает только пары строк, для которых выполнено условие соединения. Район без станций в результат не попал (12 строк, рисунок 3).

```
SELECT d.id AS district_id, d.name AS district, s.name AS station
FROM district AS d
INNER JOIN station AS s ON s.district_id = d.id
ORDER BY d.id, s.id;
```

```
PS ...\pz2-relations-joins-generation> npm start
 district_id |  district   |      station
-------------+-------------+-------------------
           1 | Центральный | Площадь Революции
           1 | Центральный | Городской сад
           2 | Приморский  | Морской вокзал
           2 | Приморский  | Приморский бульвар
           3 | Заречный    | Заречье
           4 | Октябрьский | Октябрьская
           5 | Ленинский   | Университет
           5 | Ленинский   | Студгородок
           6 | Советский   | Советская
           7 | Кировский   | Парк Победы
           8 | Московский  | Московские ворота
          10 | Северный    | Северное депо
(12 rows)
```

Рисунок 3 – Результат INNER JOIN

4.2 LEFT JOIN

Сохраняет все строки левой таблицы; для района «Красногорский» правая часть пуста (NULL), всего 13 строк (рисунок 4). Так находят объекты без связанных записей.

```
SELECT d.id AS district_id, d.name AS district, s.name AS station
FROM district AS d
LEFT JOIN station AS s ON s.district_id = d.id
ORDER BY d.id, s.id;
```

```
PS ...\pz2-relations-joins-generation> npm start
 district_id |   district    |      station
-------------+---------------+-------------------
           1 | Центральный   | Площадь Революции
           1 | Центральный   | Городской сад
           2 | Приморский    | Морской вокзал
           2 | Приморский    | Приморский бульвар
           3 | Заречный      | Заречье
           4 | Октябрьский   | Октябрьская
           5 | Ленинский     | Университет
           5 | Ленинский     | Студгородок
           6 | Советский     | Советская
           7 | Кировский     | Парк Победы
           8 | Московский    | Московские ворота
           9 | Красногорский |
          10 | Северный      | Северное депо
(13 rows)
```

Рисунок 4 – Результат LEFT JOIN

4.3 FULL OUTER JOIN

Сохраняет строки обеих таблиц, у которых нет пары. Станции соединяются с поездками по станции конца: в результате 14 пар, станция «Заречье», на которую никто не приезжал (справа NULL), и две незавершённые поездки без станции (слева NULL) – всего 17 строк (рисунок 5).

```
SELECT s.id AS station_id, s.name AS station, r.id AS rental_id,
       CASE WHEN s.id IS NULL THEN 'поездка не завершена'
            WHEN r.id IS NULL THEN 'на станцию никто не приезжал'
            ELSE 'пара найдена' END AS match_kind
FROM station AS s
FULL OUTER JOIN rental AS r ON r.end_station_id = s.id
ORDER BY s.id NULLS LAST, r.id;
```

```
PS ...\pz2-relations-joins-generation> npm start
 station_id |      station       | rental_id |          match_kind
------------+--------------------+-----------+------------------------------
          1 | Площадь Революции  |         4 | пара найдена
          1 | Площадь Революции  |        13 | пара найдена
          2 | Городской сад      |         1 | пара найдена
          3 | Морской вокзал     |         9 | пара найдена
          4 | Приморский бульвар |         3 | пара найдена
          5 | Заречье            |           | на станцию никто не приезжал
          6 | Октябрьская        |         5 | пара найдена
          7 | Университет        |        10 | пара найдена
          7 | Университет        |        11 | пара найдена
          8 | Студгородок        |         2 | пара найдена
          9 | Советская          |         7 | пара найдена
         10 | Парк Победы        |         6 | пара найдена
         11 | Московские ворота  |         8 | пара найдена
         11 | Московские ворота  |        14 | пара найдена
         12 | Северное депо      |        12 | пара найдена
            |                    |        15 | поездка не завершена
            |                    |        16 | поездка не завершена
(17 rows)
```

Рисунок 5 – Результат FULL OUTER JOIN

4.4 CROSS JOIN

Декартово произведение: каждая строка левой таблицы сочетается с каждой строкой правой, условие соединения не нужно. Используется для перебора комбинаций, например «район × тариф». Мощность результата равна произведению мощностей таблиц: без фильтра 10 × 10 = 100 строк; в запросе выбраны два района и три тарифа, поэтому 2 × 3 = 6 строк (рисунок 6).

```
SELECT d.name AS district, t.code AS tariff
FROM district AS d
CROSS JOIN tariff AS t
WHERE d.id <= 2 AND t.id <= 3
ORDER BY d.id, t.id;
```

```
PS ...\pz2-relations-joins-generation> npm start
  district   |  tariff
-------------+-----------
 Центральный | MIN_BASE
 Центральный | MIN_NIGHT
 Центральный | STUDENT
 Приморский  | MIN_BASE
 Приморский  | MIN_NIGHT
 Приморский  | STUDENT
(6 rows)
```

Рисунок 6 – Результат CROSS JOIN

5 Массовая генерация данных

Данные генерируются средствами самой СУБД: generate_series порождает нужное число строк, random() даёт случайные значения, массивы справочников задают правдоподобные имена и адреса. Параметры генерации задаёт файл 05_generate.sql, сам алгоритм – файл generate.sql:

```
\set seed 0.2026
\set stations 60
\set bicycles 600
\set clients 2000
\set rentals 10000
\ir generate.sql
```

План генерации приведён в таблице 3: сначала создаются родительские сущности, затем дочерние с корректными внешними ключами.

Таблица 3 – План генерации данных

Шаг	Таблица	Строк	Способ
1	station	60	районы по кругу (без «Красногорского»), адрес из справочника улиц, случайные координаты и дата открытия
2	bicycle	600	серийный номер VG-000001…, вид (городской 55 %, электро 20 %, грузовой 10 %, подростковый 15 %), статус «свободен» у 70 %
3	client	2 000	имя и фамилия из справочников с согласованием по роду, уникальные телефон и e-mail, 15 % приглашены ранее зарегистрированными

Шаг	Таблица	Строк	Способ
4	client_wallet	2 000	кошелёк каждому новому клиенту со случайным остатком
5	client_favorite_station	4 770	у 60 % клиентов от одной до четырёх избранных станций
6	rental	10 000	случайные клиент, велосипед и станции; длительность – экспоненциальное распределение; тариф выбирается из действовавших на дату
7	payment	8 591	по завершённым поездкам со стоимостью больше нуля: способ и статус по заданным долям

Использованные приёмы:

– setseed(0.2026) – зерно генератора: при повторном запуске получаются те же данные, результаты воспроизводимы;

– CTE с ключевым словом MATERIALIZED – случайные значения вычисляются один раз на строку и не пересчитываются при повторных обращениях;

– CROSS JOIN LATERAL – выбор тарифа для каждой поездки среди действовавших на её дату;

– стоимость поездки вычисляется по тарифу: цена старта плюс платные минуты, умноженные на цену минуты; поездки последних часов остаются незавершёнными (станция конца, время окончания и стоимость – NULL), как требует ограничение ck_rental_complete;

– вставка выполняется одной транзакцией, после неё команда ANALYZE обновляет статистику для планировщика.

Результат генерации – на рисунке 7: добавлены 60 станций, 600 велосипедов, 2 000 клиентов и 10 000 поездок; всего в базе 28 139 записей, из них 10 016 поездок, поэтому критерий «не менее 10 000 записей» выполнен.

```
PS ...\pz2-relations-joins-generation> npm start


BEGIN
INSERT 0 60
INSERT 0 600
INSERT 0 2000
INSERT 0 2000
INSERT 0 4770
INSERT 0 10000
INSERT 0 8591
COMMIT
ANALYZE
       table_name        | rows
-------------------------+-------
 district                |    10
 station                 |    72
 bicycle                 |   614
 client                  |  2012
 tariff                  |    10
 rental                  | 10016
 payment                 |  8603
 client_wallet           |  2012
 client_favorite_station |  4790
 ИТОГО                   | 28139
(10 rows)
```

Рисунок 7 – Результат генерации

6 Типовой запрос приложения

Сценарий: личный кабинет клиента «история моих поездок»: станции начала и конца, тариф, стоимость и статус оплаты, свежие поездки сверху. Используются связи rental → station (дважды), rental → tariff и rental → payment. Станция конца и платёж подключены через LEFT JOIN, потому что у незавершённой поездки их ещё нет. Клиент для демонстрации выбирается автоматически – с наибольшим числом поездок.

```
SELECT client_id AS demo_client
FROM rental
GROUP BY client_id
ORDER BY count(*) DESC, client_id
LIMIT 1 \gset

SELECT :demo_client AS client_id,
       (SELECT last_name || ' ' || first_name FROM client WHERE id = :demo_client) AS client;

SELECT r.id                                   AS rental_id,
       to_char(r.started_at, 'YYYY-MM-DD HH24:MI') AS started,
       ss.name                                AS from_station,
       es.name                                AS to_station,
       t.code                                 AS tariff,
       r.distance_km                          AS km,
       r.total_cost                           AS cost,
       coalesce(p.status, 'нет платежа')      AS payment
FROM rental AS r
JOIN station AS ss ON ss.id = r.start_station_id
LEFT JOIN station AS es ON es.id = r.end_station_id
JOIN tariff AS t ON t.id = r.tariff_id
LEFT JOIN payment AS p ON p.rental_id = r.id
WHERE r.client_id = :demo_client
  AND r.started_at >= date '2025-01-01'
ORDER BY r.started_at DESC
LIMIT 8;
```

Результат (рисунок 8): восемь последних поездок клиента; у поездок по бесплатному тарифу PROMO_1 платежа нет.

```
PS ...\pz2-relations-joins-generation> npm start
 client_id |    client
-----------+--------------
       963 | Попова Милана
(1 row)

 rental_id |     started      |  from_station  | to_station  |  tariff  |  km  |  cost  |   payment
-----------+------------------+----------------+-------------+----------+------+--------+-------------
      8676 | 2026-06-11 19:02 | Станция №59    | Станция №17 | MIN_BASE | 4.76 | 218.50 | succeeded
      8623 | 2026-06-05 15:19 | Морской вокзал | Станция №68 | PROMO_1  | 5.53 |   0.00 | нет платежа
      8224 | 2026-04-28 18:26 | Станция №51    | Станция №42 | STUDENT  | 1.81 |  39.50 | succeeded
      6635 | 2025-11-27 22:14 | Станция №72    | Станция №61 | PROMO_1  | 3.24 |   0.00 | нет платежа
      6591 | 2025-11-24 18:49 | Советская      | Станция №29 | WEEKEND  | 9.15 | 160.00 | succeeded
      6176 | 2025-10-17 05:41 | Станция №42    | Станция №17 | ELECTRO  | 5.65 | 282.60 | succeeded
      6048 | 2025-10-04 20:00 | Станция №60    | Станция №68 | STUDENT  | 1.11 |  15.00 | succeeded
      5055 | 2025-07-01 03:45 | Студгородок    | Станция №65 | WEEKEND  | 6.92 | 170.00 | succeeded
(8 rows)
```

Рисунок 8 – История поездок клиента

7 Выводы

1 Для каждой пары таблиц определён тип связи (1 : 1, 1 : N, M : N, рекурсивная) и выбрано действие при удалении: RESTRICT для справочников и истории, CASCADE для зависимых данных, SET NULL для мягких ссылок.

2 Связи реализованы в DDL внешними ключами, ограничениями UNIQUE, совпадением PRIMARY KEY и FOREIGN KEY (1 : 1) и

промежуточной таблицей с составным ключом (M : N); в схеме 12 внешних ключей.

3 Продемонстрированы INNER, LEFT, FULL OUTER и CROSS JOIN: на подготовленных данных видно, какие строки каждый вид соединения сохраняет (12, 13, 17 и 6 строк) или отсекает.

4 Сгенерировано 28 139 записей (10 016 поездок) встроенными средствами PostgreSQL воспроизводимым способом; данные согласованы с ограничениями и формулами предметной области.

5 Выполнен типовой запрос приложения с несколькими соединениями; полученные данные послужат основой для нагрузочных тестов в практическом задании №3.